

Relatório 18 - Prática: Pipelines de Dados II - Airflow (II)

Pedro de Oliveira Pinto

1. Introdução ao Apache Airflow

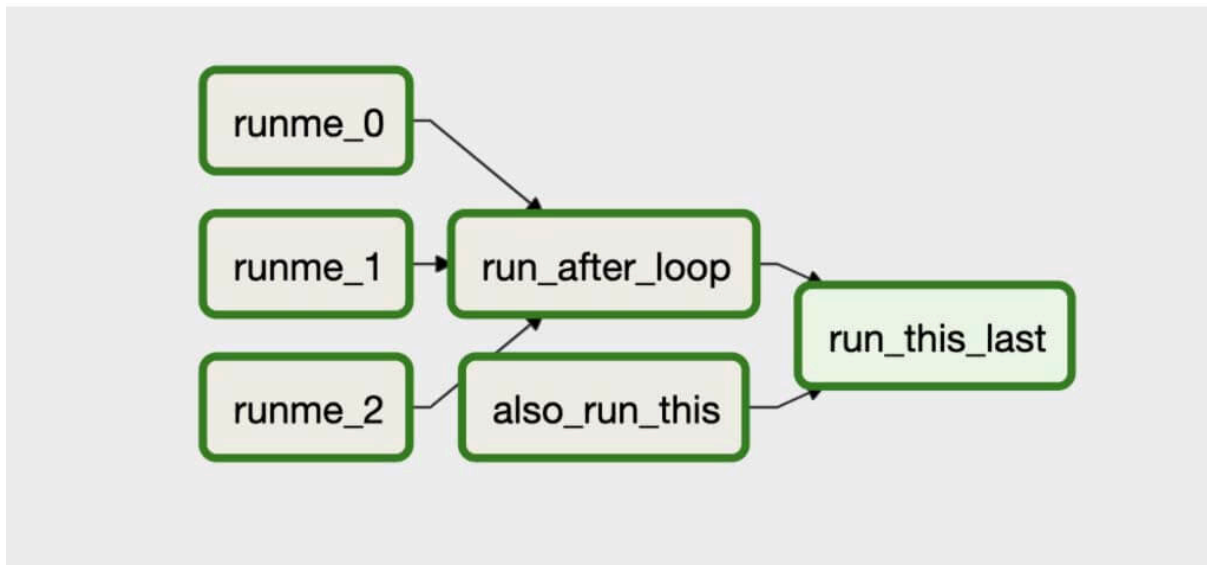
O Apache Airflow é uma plataforma de código aberto desenvolvida para programar, orquestrar e monitorar fluxos de trabalho (workflows) de maneira programática e escalável, sendo amplamente utilizado para automatizar processos complexos em áreas como engenharia de dados, machine learning e operações de infraestrutura.

2. Principais Características

2.1. Workflows como Código

Os fluxos de trabalho são definidos em Python, permitindo flexibilidade e integração com bibliotecas externas.

Utiliza DAGs (Directed Acyclic Graphs) para representar tarefas e dependências de forma visual e estruturada.

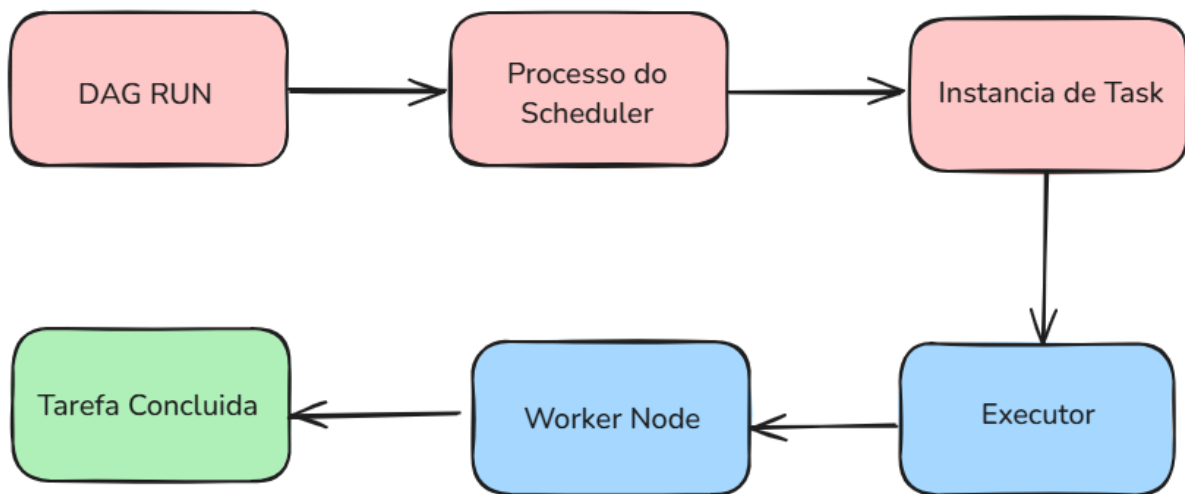


2.2. Componentes Principais

- Operadores:
 - BashOperator: Executa comandos em shell.
 - PythonOperator: Executa funções Python.
 - EmailOperator: Envia notificações por e-mail.
 - Outros: DockerOperator, SparkSubmitOperator, etc.

- Sensors: Monitoram condições externas antes de liberar a execução de tarefas (e.g., FileSensor, HttpSensor).
- Scheduler: Orquestra a execução de tarefas com base em triggers temporais ou eventos.
- Executors:
 - LocalExecutor: Executa tarefas localmente (para ambientes pequenos).
 - CeleryExecutor: Distribui tarefas em workers remotos (escalável).
 - KubernetesExecutor: Executa tarefas em pods do Kubernetes.
 - Webserver: Interface gráfica para monitorar DAGs, visualizar logs e gerenciar execuções.
 - Scheduler: Orquestra a execução de tarefas com base em triggers temporais ou eventos.

A seguinte imagem ilustra como estes componentes se comunicam entre si durante a execução de um DAG:



2.3. Outras Funcionalidades Importantes

- Retries e Alertas:
 - Configuração de políticas de repetição automática para tarefas falhas.
 - Integração com Slack, PagerDuty ou e-mail para notificações.
- Paralelismo:
 - Execução simultânea de tarefas usando workers distribuídos.
 - Controle de concorrência por DAG ou tarefa

- Backfilling: Reprocessamento de dados históricos para corrigir ou atualizar informações.
- Integrações:
 - Cloud (AWS, GCP, Azure).
 - Bancos de dados (PostgreSQL, MySQL, BigQuery).
 - Frameworks (Spark, Hadoop, dbt).

3. Arquitetura do Airflow

3.1. Componentes da Arquitetura

- Metadados Database: Armazena informações sobre DAGs, tarefas e execuções (usando PostgreSQL, MySQL ou SQLite).
- Scheduler: Analisa DAGs e agenda tarefas com base em intervalos definidos.
- Workers: Executam as tarefas (gerenciados pelo Executor).
- Webserver: Fornece uma UI para visualização de DAGs, logs e status.

4. Melhores Práticas

- Organização de DAGs:
 - Dividir DAGs complexos em subfluxos menores.
 - Evitar dependências cíclicas.
- Gestão de Recursos:
 - Usar Pools para limitar concorrência de tarefas críticas.
 - Configurar timeouts para evitar tarefas travadas.
- Versionamento:
 - Armazenar DAGs em repositórios Git para controle de mudanças.
 - Monitoramento Proativo:
 - Utilizar alertas para falhas e métricas de desempenho.

Conclusão

O Apache Airflow é uma ferramenta poderosa para a automação e orquestração de workflows, destacando-se pela flexibilidade de definir

processos como código e pela capacidade de integrar-se com diversas tecnologias modernas. Sua arquitetura modular, aliada a recursos como paralelismo, retries automáticos e uma interface gráfica intuitiva, torna-o ideal para cenários que vão desde pipelines simples até sistemas distribuídos complexos.